KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

INSURANCE POLICY MANAGEMENT

Submitted by

GOPIKA R

25BIT016

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **I**

DECLARATION

I hereby declare that the project entitled "**INSURANCE POLICY MANAGEMENT"** submitted in partial fulfillment of the requirements for the course **[B.Sc.IT I-A]** is a **record of my original work** carried out under the guidance of **[DHILEEBAN A]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

CERTIFICATE

This is to certify that the project entitled **"INSURANCE POLICY MANAGEMENT"** is a **bona fide work** carried out by **[GOPIKA R] [I A]**, **[B.Sc.INFORMATION TECHNOLOGY]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[Course Name]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place:

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Insurance Policy Management System** is a Java-based application designed to automate the management of insurance policies. In many small insurance offices and agencies, policy records are maintained manually, which can lead to calculation errors, delays in service, and inefficient record management. This project aims to simplify policy handling by providing an automated system that registers policies, calculates premiums, and generates accurate premium receipts.

The system allows users to add new policies by entering details such as policy number, holder name, age, policy type, sum assured, and policy term, and automatically calculates the annual premium based on the policy type and the age of the policyholder. The application is developed using **Core Java programming concepts**, including classes, objects, methods, conditional statements, loops, and collections such as ArrayList. The project is implemented as a **console-based, menu-driven application**.

The main objective of the system is to improve policy management efficiency, reduce human errors, and provide faster service to customers. The system maintains a clear record of each policy and generates a structured receipt that displays the policy details, status, and the annual premium payable.

This project demonstrates the practical implementation of **object-oriented programming concepts in Java** and provides a basic foundation for developing more advanced insurance management systems in the future.

1. INTRODUCTION

In the insurance industry, efficient policy management systems play a crucial role in maintaining customer records and ensuring customer satisfaction. Traditionally, many small insurance offices rely on manual methods, where staff maintain policy details in registers or paper files and calculate premiums using calculators. This manual process is time-consuming and can often lead to calculation errors, delays in service, and difficulty in maintaining proper policy records.

With the advancement of computer technology, automated management systems have become essential for improving efficiency and accuracy in insurance services. A computerized system helps staff quickly register policies, calculate premiums automatically, and maintain organized records of policies and their current status.

The **Insurance Policy Management System** is a Java-based application designed to automate the process of policy registration, status tracking, and premium receipt generation. The system allows the user to add policies, update their status, search for a policy using its policy number, and generate a receipt showing the annual premium payable.

This project is developed using **Core Java programming concepts** such as classes, objects, methods, conditional statements, loops, and collections. The application is implemented as a **console-based, menu-driven program**.

The primary goal of this system is to reduce manual effort, improve accuracy, and provide faster service to customers. This project also demonstrates how Java programming can be applied to develop simple real-world business applications.

1.1 Objective

The main objective of the **Insurance Policy Management System** is to develop a simple and efficient application using Core Java that automates the process of policy registration, status management, and premium calculation in insurance offices.

The specific objectives of this project are:

- To design and develop a **Java-based insurance policy management system**.
- To automate the **calculation of insurance premium based on policy details**.
- To reduce **manual errors in policy records and calculations**.
- To provide a **user-friendly system for insurance staff** to manage policies quickly.
- To demonstrate the implementation of **Core Java programming concepts** such as classes, objects, methods, loops, and conditional statements.
- To generate a **clear and organized premium receipt for customers**.

1.2 Overview

The **Insurance Policy Management System** is a menu-driven application developed using Java programming language. The system provides a digital platform where insurance staff can register policies, update their status, and generate premium receipts automatically.

The system displays a menu of available operations. The user can add a new policy by entering the policy details, search for an existing policy using the policy number, or update the status of a policy. The program calculates the annual premium based on the policy type, sum assured, and age of the policyholder.

The system helps simplify policy management by eliminating manual calculations and reducing the chances of errors. The application is implemented as a **console-based program** using Java collections to store policy records.

This project demonstrates the practical use of Java programming in developing real-world applications for small business environments such as insurance agencies.

1.3 Features

The **Insurance Policy Management System** includes the following main features:

• **Policy Registration** – Allows the addition of new policies with details such as policy number, holder name, age, policy type, sum assured, and term.

• **Policy Search** – Allows the user to search for a policy using its unique policy number and view the complete details.

• **Status Update** – Allows the status of a policy to be changed to Active, Renewed, Lapsed, or Claimed.

• **Premium Calculation** – Automatically calculates the annual premium based on the policy type, sum assured, and age of the policyholder.

• **Receipt Generation** – Generates a detailed premium receipt showing policy details, status, and the annual premium.

• **User-Friendly Interface** – Provides a simple and easy-to-use menu-driven interface.

• **Fast Processing** – Reduces the time required to manage policies and generate receipts.

2. PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the requirements of the system are identified. This stage helps in understanding how the system should work and what features are required to solve the problem efficiently.

The analysis phase focuses on identifying the current issues in the existing system and proposing a computerized solution that improves efficiency and accuracy.

2.1 Problem Definition

In many small insurance offices and agencies, policy details are often maintained manually using registers, paper files, or simple calculators. This traditional method creates several challenges.

Some common problems include:

- Errors in premium calculation
- Time-consuming record keeping
- Difficulty in searching for policy details
- Lack of proper policy records
- Delay in updating policy status

Manual policy management also increases the workload for insurance staff and can lead to customer dissatisfaction due to delays or incorrect premium amounts.

Therefore, there is a need for an automated system that can quickly register policies, calculate premiums, maintain accurate records, and generate receipts.

The **Insurance Policy Management System** is designed to solve these problems by providing a computerized solution that automates the policy management process.

2.2 Project Overview

The **Insurance Policy Management System** is a Java-based application developed to simplify the management of insurance policies in insurance offices and agencies.

The system provides a menu-driven interface where users can add policies, search for policies, update policy status, and generate premium receipts. The system automatically calculates the premium for each policy and stores the records for later use.

The system includes the following functionalities:

• Adding new policies with complete details
• Searching a policy using the policy number
• Updating the policy status
• Calculating the annual premium automatically
• Generating the premium receipt for the customer

This project is implemented using **Core Java programming concepts** as a **console-based, menu-driven program**.

2.3 Scopes and Limitations

Scope

The scope of the **Insurance Policy Management System** includes the following:

• Registering and storing policy records
• Searching policy details using the policy number
• Updating the status of policies
• Calculating the premium automatically
• Generating a detailed premium receipt
• Demonstrating the practical use of Java programming concepts

The system can be used in:

• Insurance agencies
• Insurance branch offices
• Banks offering insurance services
• Financial advisory firms
• Small insurance businesses

Limitations

The system has certain limitations:

• Records are stored only in memory and are lost when the program closes
• Online premium payment is not supported
• Claim processing and renewal reminders are not included
• Security features such as login authentication are not included

3. SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The **Insurance Policy Management System** is designed as a menu-driven application where users can add policies, update their status, search policy details, and generate a premium receipt automatically.

The system is divided into several modules to perform specific tasks. Each module is responsible for handling a particular function of the system.

3.1 Modules

The main modules of the **Insurance Policy Management System** are explained below:

1. Menu Display Module

This module is responsible for displaying the list of available operations to the user. The menu helps the user select the required operation.

Example Menu:

1. Add Policy
2. Update Policy Status
3. Search Policy
4. Generate Premium Receipt
5. Exit

The menu is displayed repeatedly until the user selects the exit option so that the user can easily choose the required operation.

2. Option Selection Module

This module allows the user to select an operation from the menu and enter the required details such as policy number, holder name, age, policy type, sum assured, and term.

Functions of this module include:

• Accepting the user's menu choice
• Accepting policy details from the user
• Storing policy records in the system

The system allows users to perform multiple operations until the user chooses to exit.

3. Premium Calculation Module

This module performs all the premium calculations.

The system calculates:

• Base premium rate for the policy type
• Age factor of the policyholder
• Annual premium amount
• Premium payable for the policy

The calculation is performed automatically using Java programming logic.

Example Calculation:

Premium = Sum Assured × Rate × Age Factor
Rate = 2% (Life), 1.5% (Health), 1.2% (Vehicle)
Age Factor = 1.0 (age 30 or below), 1.2 (age 31 to 45), 1.5 (above 45)
Example: 300000 × 0.015 × 1.2 = ₹5400

4. Receipt Generation Module

This module generates the premium receipt for the customer.

The receipt includes:

• Policy number
• Holder name
• Age
• Policy type
• Sum assured
• Term, status, and annual premium

Example Receipt Output:

```
========= PREMIUM RECEIPT =========
Policy No      : POL102
Holder Name    : Arun
Age            : 35
Policy Type    : Health
Sum Assured    : Rs.300000.0
Term (Years)   : 5
Status         : Active
Annual Premium: Rs.5400.0
==================================
```

5. Exit Module

This module allows the user to exit the application safely after completing the required operations.

When the user selects the exit option, the system terminates the program.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **Insurance Policy Management System**, a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on the needs of insurance policy management.

The main problems identified in manual policy management systems are:

• Manual calculations may lead to premium errors.
• Searching policy records manually takes more time.
• Difficult to handle a large number of policies efficiently.
• Lack of automated records of policy status.

Based on these problems, the system requirements were defined to develop an automated application that simplifies insurance policy management.

4.2. System Design

In this stage, the overall structure of the system is designed before coding begins.

The system is divided into modules such as:

• Menu Display Module
• Option Selection Module
• Premium Calculation Module
• Receipt Generation Module
• Exit Module

The program flow and structure are designed so that users can easily interact with the system.

4.3. Program Development

After designing the system, the program is implemented using the **Java programming language**.

The following steps were followed during development:

• Creation of classes and methods
• Implementation of menu-driven logic
• Writing code for premium calculation
• Handling user input
• Displaying the policy details and receipt

Core Java concepts such as **classes, objects, loops, conditional statements, and methods** were used during development.

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

Different test cases were used to check the functionality of the system.

Example Test Cases:

Test Case	**Expected Result**
Add a new policy	Policy added successfully
Search policy (valid / invalid number)	Details displayed / "Policy not Found" message displayed
Update policy status	Status updated successfully
Generate premium receipt	Correct premium displayed

This phase ensures that the program runs without errors and produces accurate results.

4.5. Result and Documentation

After successful testing, the results of the system are documented. The system registers policies, updates their status, and generates the premium receipt based on the policy details entered.

The results are demonstrated through program execution and screenshots showing the input and output of the system.

Proper documentation is prepared to explain the design, implementation, and working of the system.

4.6. Algorithm

Algorithm for the **Insurance Policy Management System**:

1. Start the program.
2. Display the menu (Add Policy, Update Status, Search Policy, Generate Receipt, Exit).
3. Read the user's choice.
4. If the choice is Add Policy, read the policy details and store them in the system.
5. If the choice is Update Status, read the policy number and the new status and update the record.
6. If the choice is Search Policy, read the policy number and display the details if found; otherwise display "Policy not Found".
7. If the choice is Generate Receipt, calculate the annual premium using the sum assured, the rate for the policy type, and the age factor.
8. Display the premium receipt to the user.
9. Repeat steps 2–8 until the user selects Exit.
10. Display the closing message.
11. End the program.

4.7. Flowchart

The flow of the **Insurance Policy Management System** is shown below:

Start
↓
Display Menu
↓
Read User Choice
↓
Perform Operation (Add / Update / Search / Generate Receipt)
↓
Exit selected?
↓
No → Return to Display Menu
Yes → End

5. SOURCE CODE

MAIN CLASS:

```
package com.example.insurance.main;

import com.example.insurance.model.Policy;
import com.example.insurance.service.PolicyService;

import java.time.LocalDate;
import java.util.Scanner;

public class InsuranceApp {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        PolicyService service = new PolicyService();

        while (true) {
            System.out.println("\n==== INSURANCE MANAGEMENT SYSTEM ====");
            System.out.println("1. Add Policy");
            System.out.println("2. View Policies");
            System.out.println("3. Check Premium Reminders");
            System.out.println("4. Delete Policy");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");

            int choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Policy ID: ");
                    int id = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Customer Name: ");
                    String name = sc.nextLine();

                    System.out.print("Policy Type: ");
                    String type = sc.nextLine();

                    System.out.print("Premium Amount: ");
                    double premium = sc.nextDouble();

                    System.out.print("Due Date (YYYY-MM-DD): ");
                    String date = sc.next();

                    Policy policy = new Policy(id, name, type, premium, LocalDate.parse(date));
                    service.addPolicy(policy);
                    break;
                case 2:
                    service.viewPolicies();
                    break;
                case 3:
                    service.checkReminders();
                    break;
                case 4:
                    System.out.print("Enter Policy ID to delete: ");
                    int deleteId = sc.nextInt();
                    service.deletePolicy(deleteId);
                    break;
                case 5:
                    System.out.println("Exiting...");
                    return;

                default:
                    System.out.println("Invalid choice!")
            }
            }
        }
    }
```

6. INPUT AND OUTPUT SCREENS

Figure 1: Adding the first policy

```
===== INSURANCE POLICY MANAGEMENT =====
1. Add Policy
2. Update Policy Status
3. Search Policy
4. Generate Premium Receipt
5. Exit
Enter your choice: 1
Enter Policy No      : POL101
Enter Holder Name    : Kowsika
Enter Age            : 20
Enter Type (Life/Health/Vehicle): Life
Enter Sum Assured    : 500000
Enter Term (Years)   : 10
Policy added successfully
```

Figure 2: Adding a second policy

```
===== INSURANCE POLICY MANAGEMENT =====
1. Add Policy
2. Update Policy Status
3. Search Policy
4. Generate Premium Receipt
5. Exit
Enter your choice: 1
Enter Policy No      : POL102
Enter Holder Name    : Arun
Enter Age            : 35
Enter Type (Life/Health/Vehicle): Health
Enter Sum Assured    : 300000
Enter Term (Years)   : 5
Policy added successfully
```

Figure 3: Searching a policy (valid policy number)

```
===== INSURANCE POLICY MANAGEMENT =====
1. Add Policy
2. Update Policy Status
3. Search Policy
4. Generate Premium Receipt
5. Exit
Enter your choice: 3
Enter Policy No : POL101
----- Policy Details -----
Policy No     : POL101
Holder Name   : Kowsika
Age           : 20
Policy Type   : Life
Sum Assured   : Rs.500000.0
Term (Years)  : 10
Status        : Active
```

Figure 4: Searching a policy (invalid policy number)

```
===== INSURANCE POLICY MANAGEMENT =====
1. Add Policy
2. Update Policy Status
3. Search Policy
4. Generate Premium Receipt
5. Exit
Enter your choice: 3
Enter Policy No : POL999
Policy not Found
```

Figure 5: Updating the policy status

```
===== INSURANCE POLICY MANAGEMENT =====
1. Add Policy
2. Update Policy Status
3. Search Policy
4. Generate Premium Receipt
5. Exit
Enter your choice: 2
Enter Policy No : POL101
Enter new status (Active/Renewed/Lapsed/Claimed): Renewed
Policy status updated successfully
```

Figure 6: Generating the premium receipt

```
===== INSURANCE POLICY MANAGEMENT =====
1. Add Policy
2. Update Policy Status
3. Search Policy
4. Generate Premium Receipt
5. Exit
Enter your choice: 4
Enter Policy No : POL102
========= PREMIUM RECEIPT =========
Policy No     : POL102
Holder Name   : Arun
Age           : 35
Policy Type   : Health
Sum Assured   : Rs.300000.0
Term (Years)  : 5
Status        : Active
Annual Premium: Rs.5400.0
==================================
```

Figure 7: Exiting the program

```
===== INSURANCE POLICY MANAGEMENT =====
1. Add Policy
2. Update Policy Status
3. Search Policy
4. Generate Premium Receipt
5. Exit
Enter your choice: 5
Thank you. Program closed.
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

The **Insurance Policy Management System** was developed to automate policy management in insurance offices. The system allows users to add policies, update their status, search policy details, and generate premium receipts automatically.

During the development process, Core Java programming concepts such as loops, conditional statements, methods, collections, and object-oriented principles were applied. The system successfully performs premium calculations and generates accurate receipts.

The system was tested with different inputs to ensure that it performs the required operations correctly. The results show that the system reduces manual effort and improves the efficiency of policy management.

7.2 Conclusion

The **Insurance Policy Management System** successfully demonstrates the practical application of Core Java programming concepts in developing a real-world business application.

The system automates policy management, reduces calculation errors, and provides faster service to customers. It simplifies insurance office operations by maintaining policy records and generating accurate premium receipts.

This project helps students understand how Java programming can be used to develop practical software systems. The project can be further enhanced by adding features such as database connectivity, graphical user interfaces, claim processing, and online payment systems.

8. REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com